CS 10C Lab 9 - Cuckoo Hashing

Prof. Ty Feng, UCR

Goal

In this lab, you will expand your knowledge about hash tables by implementing another variant of hashing, namely the *Cuckoo Hashing*.

Cuckoo Hashing

Cuckoo hashing is one of the many forms of hash tables which arose from the motivation of accomplishing a worst case O(1) look up time. Almost all of the other variants of hash tables you are seeing in lecture have an amortized O(1) look up time.

In cuckoo hashing, suppose we have N items. We maintain two tables, each more than half empty, and we have two independent hash functions that can assign each item to a position to each table. Cuckoo hashing maintains the invariant that an item is always stored in one of the two tables.

The cuckoo hashing algorithm begins by trying to insert an item into the first table. If there the location at the table is empty, then it inserts the item as usual. If there is a collision, then the previously inserted element at that position is rehashed and moved to the second table, and the item currently being inserted is placed in the first table instead.

The swapping of two elements from one table to an other is the collision strategy of the cuckoo hashing. There are no chains, or no probing (linear, or quadratic) strategies. If there is a collision, we simply try to move an element to the second table.

Data Structure

Assume the following API for the cuckoo hashing implementation:

```
template <typename K>
class CuckooHashing {
public:
  // Check if @key exists in one of the two hash tables
  bool Contains(const K& key);

  // Insert @key into any one of the hash tables
  void Insert(const K& key);

  // Remove @key from one of the tables if it exists
  void Remove(const K& key);
```

```
  // Print hashtable's contents
  void PrintTable();

private:
  ...
};
```

Q 1.1

Add necessary private methods and/or variables to:

1. Declare and initialize two hash tables based on an initial capacity
2. Compute the positions of a given item based on two independent hash functions

Q 1.2

Now, implement the two independent hash functions. The function prototypes are as follows:

```
size_t Hash1(const K& key);
size_t Hash2(const K& key);
```

Implementing the API

Q 2.1

Implement the `Contains()` method. It should check if the key exists in one of the two hash tables.

Q 2.2

Now, implement the `Insert()` method. The cuckoo hashing algorithm starts by checking if the element is already inserted. If it isn't, then it proceeds to insert it in the first table. If there is a collision, then the positions are swapped repeatedly until there are no more collisions remaining. One way to think about repeatedly swapping between tables is recursion.

It may also help to keep track of the table you are inserting into. Since the API doesn't provide you with this, you might want to write your own private recursive helper method to assist with this.

Q 2.3

Complete the insertion algorithm by implementing the recursive helper method.

The function prototype is as follows:

```
// Recursively insert @key into table @id
void InsertRecur(const K& key, int id);
```

Q 2.4

Because of the nature of using two hash tables and the collision strategy implemented, for a certain number of elements inserted, the cuckoo hashing algorithm tries to keep both the tables half empty. This can be thought of as a load factor of 0.5.

Usually, if we insert N elements less than or equal to the capacity, we can expect a nice distribution of these items within these two tables. However, if we insert more than that, we need to rehash.

Modify method `Insert()` to account for the resizing, and write the `Resize()` private method that doubles the size of the hashtables and rehashes the entire collection.

Q 2.5

Finally, implement the `Remove()` method that resets the position of the table that the item is present in.